This content is owned and created by Henri LEPIC • if you want to use it please contact me • henrilepic@gmail.com

25

keywords in go and more :)

Number of keywords is an approximate measure of complexity :

C#	77
PHP	74
C++	72
Rust	52
Java	50
Ruby	41
Pascal	35
C	32
Python	31
JavaScript	26
Go	25

package

keywords

n°1

```go
package main
```

```go
package fmt
```

```go
package service
```

```go
package service_test
```

Nommer ses packages

import

keywords

n°2

Import : simple

```
1   import "fmt"
2   import "image/png"
3   import "github.com/go-gorm/gorm"
4   import "github.com/spf13/viper"
5   import "github.com/mycompagny/projectname/model"
6   import "github.com/mycompagny/projectname/control"
7
8
9
10
11
```

Import : groupé

L'ordre des imports idiomatique est d'abord la librairie standard, ensuite les projets externes et enfin les packages interne. Remarquez les espaces entre chaque groupes.

```
1   import (
2       "fmt"
3       "image/png"
4
5       "github.com/go-gorm/gorm"
6       "github.com/spf13/viper"
7
8       "github.com/mycompagny/projectname/model"
9       "github.com/mycompagny/projectname/control"
10  )
11
```

Import : redéfinition

```
1   import (
2       // possibilité d'être non utilisé.
3       _ "log"
4
5       // les fonctions ,variables et constantes exportés
6       // peuvent être appelés directement.
7       . "strconv"
8
9       // le nom du package est renommé.
10      format "fmt"
11  )
```

play

var

keywords

n°3

variable : création

```
1  var msg string
2  msg = "my message"
3  fmt.Printf("%T - %v - %v", msg, msg, &msg) // ?
4
5
6
7
8
```

play

variable : création et initialisation

```go
var msg1 string = "Hello"

// same same
msg2 := "Gophers"

fmt.Printf("%T - %v", msg1, msg1) // ?
fmt.Printf("%T - %v", msg2, msg2) // ?

```

play

variable : création multiple sur une ligne

```
1  msg1, msg2 := "Hello", "Gophers"
2
3  fmt.Printf("%T - %v", msg1, msg1) // ?
4  fmt.Printf("%T - %v", msg2, msg2) // ?
5
6
7
8
```

play

variable : scope de variables

```
1  var msg string = "my message"
2  {
3      var msg string = "egassem ym"
4      fmt.Println(msg) // ?
5  }
6  fmt.Println(msg) // ?
7
8
```

play

const

keywords

n°4

constante : type

```
1   const DefaultName = "Doe" // texte
2   const CodeNotFound = 404 // entiers
3   const PI = 3.1415 // flotant
4   ...
5   fmt.Printf("%T : %v", DefaultName, DefaultName) // ??
6   fmt.Printf("%T : %v", CodeNotFound, CodeNotFound) // ??
7   fmt.Printf("%T : %v", PI, PI) // ??
8
9
10
11
```

play

constante : scope

```
1   const DefaultName = "Doe" // texte
2   const CodeNotFound = 404 // entiers
3   const PI = 3.1415 // flotant
4   ...
5   fmt.Printf("%T : %v", DefaultName, DefaultName) // ??
6   fmt.Printf("%T : %v", CodeNotFound, CodeNotFound) // ??
7   fmt.Printf("%T : %v", PI, PI) // ??
8
9
10
11
```

play

constante : alias de type

```
1   type Periodic int
2   const Ag Periodic = iota + 1
3   fmt.Printf("%T : %v", Ag, Ag) // main.Periodic : 1
4
5
6
7
8
9
10
11
```

play

constante : déclaration groupée

```
1   type Periodic int
2   const (
3       Ag Periodic = 1
4       La Periodic = 2
5       Ce Periodic = 3
6       Gd Periodic = 4
7       Pt Periodic = 5
8       Au Periodic = 6
9   )
10  …
11  fmt.Printf("%T : %v", Ag, Ag) // main.Periodic : 1
```

play

constante : déclaration avec iota

```
1   type Periodic int
2   const (
3       Ag Periodic = iota + 1 // type Periodic de valeur 1
4       La // .. 2
5       Ce // .. 3
6       Gd // .. 4
7       Pt // .. 5
8       Au // .. 6
9   )
10  …
11  fmt.Printf("%T : %v", Ag, Ag) // main.Periodic : 1
```

play

map

keywords

n°5

```
m := make(map[string]int)
m["keyA"] = 0
m["keyB"] = 1
m["keyC"] = 2
```

création classique

```
m := map[string]int{
    "keyA":0,
    "keyB":1,
    "keyC":2,
}
```

création littéral

```
v := m["keyA"]

// or
v, ok := m["keyA"]
if ok {
    print(v)
}
```

récupération d'une valeur

```
delete(m, "keyA")
```

destruction d'une clée valeur

play

Autre type de collection

no keywords

?

slice : création

```
1   var listA []string = []string{"A","B","C"}
2   // same as
3   listB := []string{"A","B","C"} // déclaration courte
4   // same as
5   listC := make([]string, 3) // initialisation
6   listC[0] = "A" // et insertion de valeur
7   listC[1] = "B"
8   listC[2] = "C"
9
10
11
```

play

slice : récupération de valeur

```
1   var list []string = []string{"A","B","C"}
2   …
3   fmt.Println(list) // tableau avec les valeurs
4   fmt.Println(list[1]) // une valeur à sa position d'index
5   fmt.Println(list[1:]) // slicing
6   fmt.Println(list[:]) // ensemble des valeurs
7
8
9
10
11
```

play

slice : ajout de valeur

```
1   var list []string = []string{"A","B","C"}
2   …
3   list = append(list, "D")
4   fmt.Println(list)
5
6
7
8
9
10
11
```

play

slice : création et ajout avancé

```
1   list := make([]string, 3, 4) // initialisation
2   fmt.Println("taille", len(list), "capacité", cap(list))
3   list[0] = "A" // et insertion de valeur
4   list[1] = "B"
5   list[2] = "C"
6   list = append(list, "D")
7   fmt.Println(list)
8   fmt.Println("taille", len(list), "capacité", cap(list))
9
10
11
```

play

for

keywords

n°6

```go
for i := 0; i < 3; i++ {
    fmt.Println(i)
}

for i := 2; i >= 0; i-- {
    fmt.Println(i)
}
```

boucle classique

```go
i := 0
for {
    fmt.Println(i)
    i++
}
```

boucle infinie

```go
a := []int{1, 2, 3}
loop, i := true, 0
for loop {
    print(a[i])
    i++
}
```

boucle conditionnelle

play

```go
list := []string{"A", "B", "C"}
for i := 0; i < len(list); i++ {
    fmt.Println(list[i])
}

for i := len(list); i >= 0; i-- {
    fmt.Println(list[i])
}
```

boucle classique

```go
i := 0
for {
    fmt.Println(i)
    i++
}
```

boucle infinie

```go
a := []int{1, 2, 3}
loop, i := true, 0
for loop {
    print(a[i])
    i++
}
```

boucle conditionnelle

play

range

keywords

n°7

```go
a := []int{1, 2, 3}
for _, v := range a {
    print(v)
}
```

boucle par copie de valeur

```go
a := []int{1, 2, 3}
for k, _ := range a {
    print(a[k])
}
```

boucle par clé

```go
a := []int{1, 2, 3}
for k := range a {
    print(a[k])
}
```

boucle par clé simple

break

keywords

n°8

```
i := 0
for {
    if i == 2{
        break
    }
    i++
}
```

continue

keywords

n°9

```go
for i := 0; i < 3; i++ {
    if i != 1 {
        fmt.Println(i)
    }
}
```

```go
for i := 0; i < 3; i++ {
    if i == 1 {
        continue
    }
    fmt.Println(i)
}
```

if

keywords

n°10

```
if true {
    print("all good")
}
```

```
if i < 10 {
    print("i is under 10")
}
```

```
if i := Action(); i > 1 {
    print(i) //
}


func Action()int{
    return 2
}
```

else

keywords

n°11

```
if true {
   print("all good")
} else {
   print("stop")
}
```

```
if i < 10 {
   print("i is under 10")
} else {
   print("i > 10")
}
```

```
if i := Action(); i > 1 {
   print(i) //
}  else {
   print("no actions")
}


func Action()int{
   return 2
}
```

```
if true {
   print("all good")
} else if false {
   print("stop")
}
```

```
if i < 10 {
   print("i is under 10")
} else if i == 0 {
   print("i == 10")
} else {
   print("i > 10")
}
```

```
if i := Action(); i > 1 {
   print(i) //
} else if i == 0 {
   print("i == 10")
} else {
   print("no actions")
}


func Action()int{
   return 2
}
```

switch

keywords

n°12

case

keywords

n°13

```go
a := 0
switch a {
case 0, 1:
    fmt.Println(0, "or", 1)
case 2 :
    fmt.Println(2)
case 3 :
    fmt.Println(3)
}
```

play

```go
a := 3
switch {
case a <= 1:
    fmt.Println(">=", 1)
case a == 2 :
    fmt.Println("==", 2)
case a >= 3 :
    fmt.Println(">=", 3)
}
```

play

```go
switch a := 0; a {
case 0, 1:
    fmt.Println(0, "or", 1)
case 2 :
    fmt.Println(2)
case 3 :
    fmt.Println(3)
}
```

play

```go
switch a := 3; {
case a <= 1:
    fmt.Println(">=", 1)
case a == 2 :
    fmt.Println("==", 2)
case a >= 3 :
    fmt.Println(">=", 3)
}
```

play

default

keywords

n°14

```go
a := 5
switch a {
case 0, 1:
    fmt.Println(0, "or", 1)
case 2 :
    fmt.Println(2)
case 3 :
    fmt.Println(3)
default :
    fmt.Println("unkown")
}
```

play

```go
a := 3
switch {
case a <= 1:
    fmt.Println(">=", 1)
case a == 2 :
    fmt.Println("==", 2)
case a > 3 :
    fmt.Println(">", 3)
default :
    fmt.Println("unkown")
}
```

play

fallthrough

keywords

n°15

```go
a := 1
switch a {
case 0, 1:
    fmt.Println(0, "or", 1)
case 2 :
    fmt.Println(2)
case 3 :
    fmt.Println(3)
}
```

play

```go
a := 1
switch a {
case 0, 1:
    fmt.Println(1)
    fallthrough
case 2 :
    fmt.Println(2)
case 3 :
    fmt.Println(3)
}
```

play

type

keywords

n°16

Créer un nouveau type

Un nouveau type se définit toujours à partir d'un type natif ou déjà existant.
Sont type d'origine peut être retrouvé grâce au cast de type.

```
1  type id int64
2  type myID id
3
4  var i myID = 999
5  fmt.Printf("%T\n", i)
6  fmt.Printf("%T\n", id(i))
7  fmt.Printf("%T\n", int64(i))
8
9
```

play

Créer un alias type

L’alias de type permet de réaliser des restructurations de code sans le casser.

```
1 type Admin = User
2
3
4
5
```

article

struct

keywords

n°17

Créer une structure

```
1 type User struct {
2     id int // champs non exporté
3     Name string // champs exporté
4 }
5
```

Créer des champs du même type sur la même ligne

```
1  type User struct {
2      id int
3      FirstName, LastName string
4  }
5
6
7
```

Composer des structures avec d’autres structures

```go
type User struct {
    id int
    Name string
    Human
}

type Human struct {
    BirthDate time.Time
}
```

Créer une structure dans une structure

```
1  type User struct {
2      id int
3      Name string
4      Human struct {
5          BirthDate time.Time
6      }
7  }
```

func

keywords

n°18

fonction : classique

```
1  func Hello (n string) {
2      fmt.Println("Hello", n)
3  }
4
5
6
7
8
```

play

fonction : anonyme

```go
func Hello (msg string){
    func (msg string){
        if len(msg) == 0{
            fmt.Println("Hi")
            return
        }
        fmt.Println("Hello", msg)
    }(msg)
}
```

play

fonction : anonyme

```go
func Hello (msg string){
    f := func (msg string){
        if len(msg) == 0{
            fmt.Println("Hi")
            return
        }
        fmt.Println("Hello", msg)
    }
    f(msg)
}
```

play

fonction : closure

```go
func Hello (n int) func (msg string){
    return func (msg string){
        if len(msg) == n {
            fmt.Println("Hi")
            return
        }
        fmt.Println("Hello", msg)
    }
}
```

play

fonction : méthode

```go
type User struct {
    name string
}

func (u User)Hello (msg string){
    fmt.Println(u.name, ":", msg)
}

```

play

fonction : méthode avec pointeur

- Permet de modifier une des valeurs de la structure.
- Évite de copier la structure à chaque appel de la méthode.

```go
type User struct {
    name string
}

func (u *User)Name (name string){
    u.name = name
}

```

play

return

keywords

n°19

retour : classique

```
1  func Hello (n string) string {
2      return “Salut ” + n
3  }
4  …
5  msg := Hello("Bob")
6
7
8
9
```

play

retour : multiple

```
1  func Hello (n string) (string, error) {
2      if len(n) == 0{
3          return “”, errors.New(“no name”)
4      }
5      return “Salut ” + n, nil
6  }
7  …
8  msg, err := Hello("Bob")
9
```

play

retour : nommés

```
1   func Hello (n string) (res string, err error) {
2       if len(n) == 0{
3           e = errors.New(“no name”)
4       }else {
5           res = “Salut ” + n
6       }
7       return
8   }
9   …
10  msg, err := Hello("Bob")
```

play

defer

keywords

n°20

defer : simple

```
1  func Hello (msg string) {
2      defer fmt.Println("Hello", msg)
3      fmt.Println("wait two secs")
4      time.Sleep(time.Second * 2)
5  }
6
7
8
```

play

defer : pile

```go
func Hello (msg string, n int) {
    defer fmt.Println(msg, n)
    if n > 0 {
        Hello(msg, n - 1) // ?
    }
}
```

play

interface

keywords

n°21

Interface de valeur

```
var i interface{}
i = 1
fmt.Printf("%T\n",i) // int
i = 2.34
fmt.Printf("%T\n",i) // float64
i = "ABCD"
fmt.Printf("%T\n",i) // string
```

play

Interface de contrat avec error

```go
type error interface{
   Error() string
}
```

```go
type ErrInternal struct{
   Name string
   code int
}

func (e ErrInternal) Error() string {
   return e.Name
}
```

```go
err := ErrInternal{
    Name:"Access",
    code:403,
}

log.Println(err) // 2021-01-21
10:36:49 Access
```

play

Interface de contrat avec stringer

```go
type Stringer interface{
    String() string
}
```

```go
type User struct{
    Name, email string
}

func (u User) String() string {
    return u.email
}
```

```go
u := User{
    Name:"Rob Pike",
    email:"r@google.com",
}

fmt.Println(u) //r@google.com
// or
fmt.Println(&u) //r@google.com
```

play

Récupérer le type primitif d'une valeur dans une interface.

```go
var i int = 1
var a interface{} = i
switch a.(type) {
case int :
    fmt.Println("int")
case float64 :
    fmt.Println("float64")
default :
    fmt.Println("unkown")
}
```

play

go

keywords

n°22

Goroutine

```
1   go doSomethingHeavy()
2   …
3   func doSomethingHeavy(){
4       time.Sleep(time.Second*2) // simulate time exec
5       fmt.Println("done")
6   }
7
8
9
10
11
```

play

chan

keywords

n°23

Canal non bufferisé

```
1  msg := make(chan string)
2
3  go func(){
4     msg <- "non buffered"
5  }()
6
   fmt.Println(<-msg)
```

play

Goroutine

```
1   msg := make(chan string)
2   go doSomethingHeavy(msg)
3   fmt.Println(<-msg)
4   …
5   func doSomethingHeavy(msg chan string){
6       time.Sleep(time.Second*2) // simulate time exec
7       msg <- "done"
8   }
9
10
11
```

play

Canal bufferisé

```
1  msg := make(chan string, 1)
2
3  msg <- "buffered"
4
5  fmt.Println(<-msg)
6
```

play

select

keywords

n°24

Select with channel

```
1   msg := make(chan string)
2   stop := time.Tick(1 * time.Second)
3   go DoSomeThingGreat(msg)
4
5   select {
6   case <- msg :
7       fmt.Println("Yeee")
8   case <- stop :
9       fmt.Println("stop")
10      return
11  }
```

goto

keywords

n°25

```
1   i := 0
2       for {
3           fmt.Println("loop",i)
4           if i == 10 {
5               fmt.Println("stopping loop")
6               goto stop
7           }
8        }
9   stop:
10  fmt.Println("loop stopped")
11
```

~~goto~~

keywords

n°25

```
1   i := 0
2   loop:
3       for {
4           fmt.Println("loop",i)
5           if i == 10 {
6               fmt.Println("stopping loop")
7               break loop
8           }
9        }
10  fmt.Println("loop stopped")
11
```

Exercice

1. Créer un fichier main.go
2. Insérer 7 mots clefs de Go.
3. lancer avec la commande **go run main.go**
4. Essayer d’avoir moins de 20 lignes de code dans votre fichier.
5. Si vous avez réussis modifiez le fichier pour avoir qu’une ligne de code.